

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №1

по дисциплине «Системы параллельной обработки данных»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.т.н., доцент ОИТ

_____ Аксенов С.В.

Цель

Получить навыки создания многоагентной среды с неопределенностью с применением многопроцессных вычислений.

Задание

Создать многоагентную среду, в которой имеются агент-доставщик полезного груза по городу. Движение агента осуществляется одним процессом. Движение других транспортных средств (ботов) контролируется другим процессом. Опционально: можно сделать для каждого бота свой процесс.

Для этого необходимо сформировать карту, в которой будет перемещаться агент и другие транспортные средства. Пример карты для одного агента в файле “citymap.txt”. Минимальные размеры карты 30 x 30.

Пояснения к карте:

0 (а также 2, 3, 4) – области на карте, куда агент может попасть (дорога).

1 – на карте отмечены препятствия, куда агент не может попасть (стены).

2 – место, с которого агент начинает движение по карте.

3 – места, с которых с определенной вероятностью выходят другие транспортные средства (вероятность этого мала, например 0.02).

4 – Область, в которую должен попасть агент.

На каждой итерации:

Агент делает одно из действий: влево, вправо, вверх, вниз или остановка. Транспортные средства тоже делают одно из этих движений. Для каждого транспортного средства (бота) генерируется движение в область, куда можно попасть (0, 2, 3, 4) при этом, если транспортное средство начинает движение из тупика, то выбирается единственное направление – выход из тупика), при достижении перекрёстка, применяется следующее условие: с вероятностью 10% бот делает остановку, а на следующий ход бот разворачивается и движется в обратном направлении, оставшиеся 90% равномерно распределяются между другими вариантами движения через перекресток, например если бот доехал до

перекрёстка с нижней клетки и есть варианты двигаться вверх, налево и направо, то с вероятностью 10% он делает остановку, а затем перемещается вниз, с вероятностью 90% становится в центр перекрёстка, а затем выбирает движение вверх с вероятностью $1/3$, направо с вероятностью $1/3$ или налево с вероятностью $1/3$. Если на развилке только два возможных пути, то вероятности выборка каждого $1/2$. Если бот достигает тупика, то он разворачивается и едет в противоположную сторону.

В случае столкновения ботов между собой или агента с ботом, происходит авария и транспортные средства удаляются. Если столкнулся агент, то мы признаём эпизод движения агента неуспешным.

Для агента доставщика нужно тоже задать разные 3 стратегии движения на развилках дорог. Можно задать единую стратегию для пересечения развилок, как у ботов, а можно задать для каждого перекрёстка свою стратегию. Пример стратегии для развилки: Для всех развилок: с вероятностью 5% доставщик делает остановку, а затем разворачивается назад и движется в противоположном направлении, с вероятностью 40% поворачивает на право, с вероятностью 30% двигается прямо, и с вероятностью 25% двигается налево. Или для развилки А: с вероятностью 70% двигается направо и с вероятностью 30% двигается прямо.

При начальном выходе из области 3 для транспортного средства (бота) генерируется число количество шагов (случайная величина от 15 до 150). В течение этого числа итераций транспортное средство двигается, а по достижению этого числа убирается из карты (припарковывается или заезжает в гараж).

Установить на карте несколько мест, из которых будет выезжать агент. Исследовать как будет изменяться среднее количество шагов, за которое агент доставит груз, а также вероятность успеха в достижении точки доставки в зависимости от скорости генерации других транспортных средств и стратегии движения.

Выполнение работы

Для проведения экспериментов была составлена карта города размером 30×30 клеток и сохранена в файл citymap.txt(Рисунок 1). Визуализация сгенерированной карты представлена на рисунке 2. Зеленые - точки спавна агента, красный - агент, желтые - точки спавна ботов, синие — боты, оранжевая — точка доставки.

```
lab1 > src > citymap.txt
1  1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
2  1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 1
3  1 0 1 1 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1 1 3 1 1 1 1 1 1 1 1 1 0 1
4  1 0 1 1 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 0 1
5  1 0 1 1 1 3 0 1 1 0 0 0 0 0 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 0 1
6  1 0 1 1 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 0 1
7  1 0 0 0 0 0 0 0 0 0 1 1 1 0 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1
8  1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 1 1 0 1 1 0 1 1 1 0 1 1 1 1 0 1
9  1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 1 1 0 1 1 0 1 1 1 0 1 1 1 1 0 1
10 1 0 0 0 0 0 0 1 1 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1
11 1 0 1 1 1 1 0 1 1 0 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1
12 1 0 1 1 1 1 0 1 1 0 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1
13 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
14 1 0 1 0 1 1 1 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1
15 1 0 1 0 1 1 1 1 1 1 1 1 1 4 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1
16 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1
17 1 1 1 0 1 1 1 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 3 1 1 1 1 0 1
18 1 1 1 0 1 1 1 1 3 1 1 1 1 0 1 1 1 1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 1
19 1 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 1
20 1 0 0 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 0 0 0 0 0 0 0 0 0 1
21 1 0 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 0 1 1 0 1
22 1 0 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 0 1 1 0 1
23 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1 1 1 0 1
24 1 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1
25 1 0 1 1 0 1 1 1 1 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 3 1 1 0 1
26 1 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1
27 1 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1
28 1 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1
29 1 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 2 1
30 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Рисунок 1. Карта города - в файле citymap.txt.

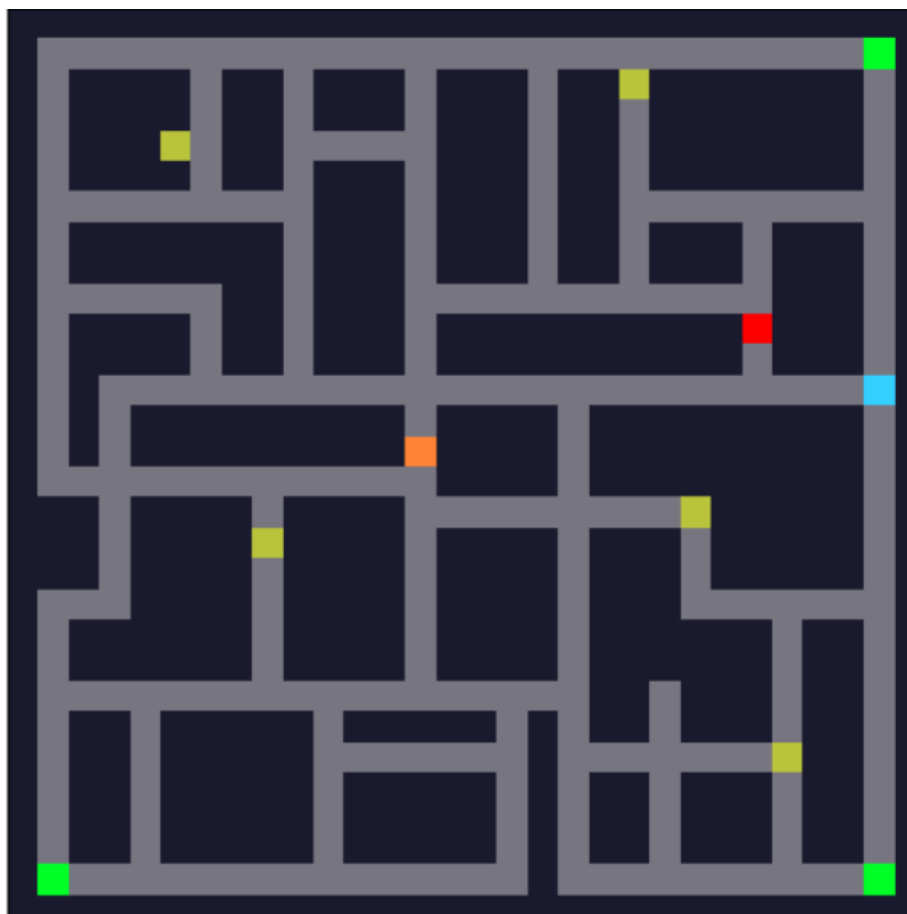


Рисунок 2. Карта города - визуализация.

Для работы с картой и пространственной ориентации был разработан **класс Navigation**. Он обеспечивает:

- Загрузку карты из текстового файла citymap.txt
- Поиск стартовых точек и цели – автоматическое определение всех клеток со значениями 2 и 4
- Определение доступных соседей – возвращает список соседних дорог, пригодных для движения
- Классификацию клеток – определяет тип текущей позиции относительно направления движения: start – начальная позиция, straight – прямая дорога (один вариант движения), dead_end – тупик (нет вариантов движения), intersection – перекресток (два и более вариантов)
- Преобразование направлений – переводит абстрактные команды (straight, left, right, back, stop) в конкретные координаты на карте с использованием векторной геометрии.

Данный модуль позволяет агенту и ботам корректно ориентироваться на карте независимо от направления подхода к перекрестку.

Далее был реализован **класс Simulator**, который реализует многопроцессную архитектуру симуляции движения. Основные компоненты:

- Разделяемое состояние – через multiprocessing.Manager создается словарь shared, содержащий карту, состояние агента, список ботов и флаги завершения эпизода.
- Синхронизация – с помощью Barrier обеспечивается одновременное выполнение шага агентом и ботами, что исключает состояние гонки данных.
- Два независимых процесса: agent_worker – управляет движением агента-доставщика и bot_worker – управляет пулом транспортных средств
- Логика движения агента: начальное движение – случайный выбор соседней дороги, в тупике – разворот в обратном направлении, на перекрестке – выбор направления согласно весам выбранной стратегии, при команде stop – следующий шаг выполняется разворот
- Логика движения ботов: спавн новых ботов в клетках со значением 3 с вероятностью spawn_prob, каждый бот имеет случайное время жизни от 15 до 150 шагов, на перекрестке с вероятностью 10% – остановка с последующим разворотом, оставшиеся 90% – равномерное распределение между доступными направлениями, в тупике – разворот
- Обработка столкновений: если агент и бот оказываются на одной клетке – авария, эпизод признается неуспешным, если два бота оказываются на одной клетке – оба удаляются. Дополнительно проверяется ситуация "разминовки" (когда объекты меняются местами за один шаг)

После был написан **класс Visualizer** предназначен для анализа результатов и создания анимации. Функциональность:

- Загрузка результатов из CSV-файла, сгенерированного экспериментатором

- Построение графиков: зависимость среднего числа шагов до цели от вероятности появления ботов (только для успешных эпизодов) и зависимость вероятности успешной доставки от вероятности появления ботов
- Генерация анимации – сохранение GIF-файла с движением агента (красный цвет) и ботов (синий цвет) на фоне цветовой карты города

Затем реализован **класс Experiment**, который обеспечивает массовый запуск симуляций для сбора статистических данных. Результаты каждого эпизода сохраняются в CSV-файл с полями: количество шагов, статус успеха, факт столкновения, вероятность спавна, стратегия, стартовая позиция.

Файл main.py является точкой входа в программу и объединяет все компоненты. Последовательность выполнения:

- Загрузка карты из файла citymap.txt
- Автоматический поиск стартовых точек и цели (по значениям 2 и 4 на карте)
- Запуск эксперимента с заданными параметрами
- Сохранение результатов в experiment_results.csv
- Построение графиков по полученным данным

Такой подход позволяет полностью автоматизировать процесс исследования – от загрузки карты до получения визуальных результатов.

В этом же файле определены параметры эксперимента:

- Вероятность спавна ботов - 0.01, 0.03, 0.05, 0.08
- Стратегии агента uniform, straight_focus, cautious
- Количество эпизодов на конфигурацию 30

Стратегии поведения агента на перекрестках:

- uniform – равномерное распределение: 25% на каждое направление (прямо, направо, налево, остановка)

- `straight_focus` – приоритет прямого движения: 50% прямо, 20% направо, 20% налево, 10% остановка
- `cautious` – осторожное поведение: 10% прямо, 10% направо, 10% налево, 70% остановка

Для удобства отладки и демонстрации был создан отдельный файл **`run_animation.py`**, позволяющий запускать анимацию для одного эпизода с выводом результата в консоль. Этот модуль удобен для быстрой проверки визуального поведения агента и ботов без запуска полного эксперимента.

Файл `episode_animation_win.gif` – анимация успешной доставки груза. Агент (отображается красным цветом) начинает движение от стартовой точки (зеленый маркер) и, избегая столкновений с транспортными средствами (синие маркеры), достигает целевой точки (оранжевый маркер).

Файл `episode_animation_accident.gif` – анимация аварийной ситуации. В данном эпизоде агент врежется в одно из транспортных средств, после чего симуляция останавливается, а эпизод признается неуспешным. На анимации отчетливо виден момент столкновения агента с ботом.

Результаты экспериментов

После завершения серии экспериментов были получены статистические данные, сохраненные в файл `experiment_results.csv`. На основе этих данных построены два графика, позволяющие оценить эффективность различных стратегий поведения агента.

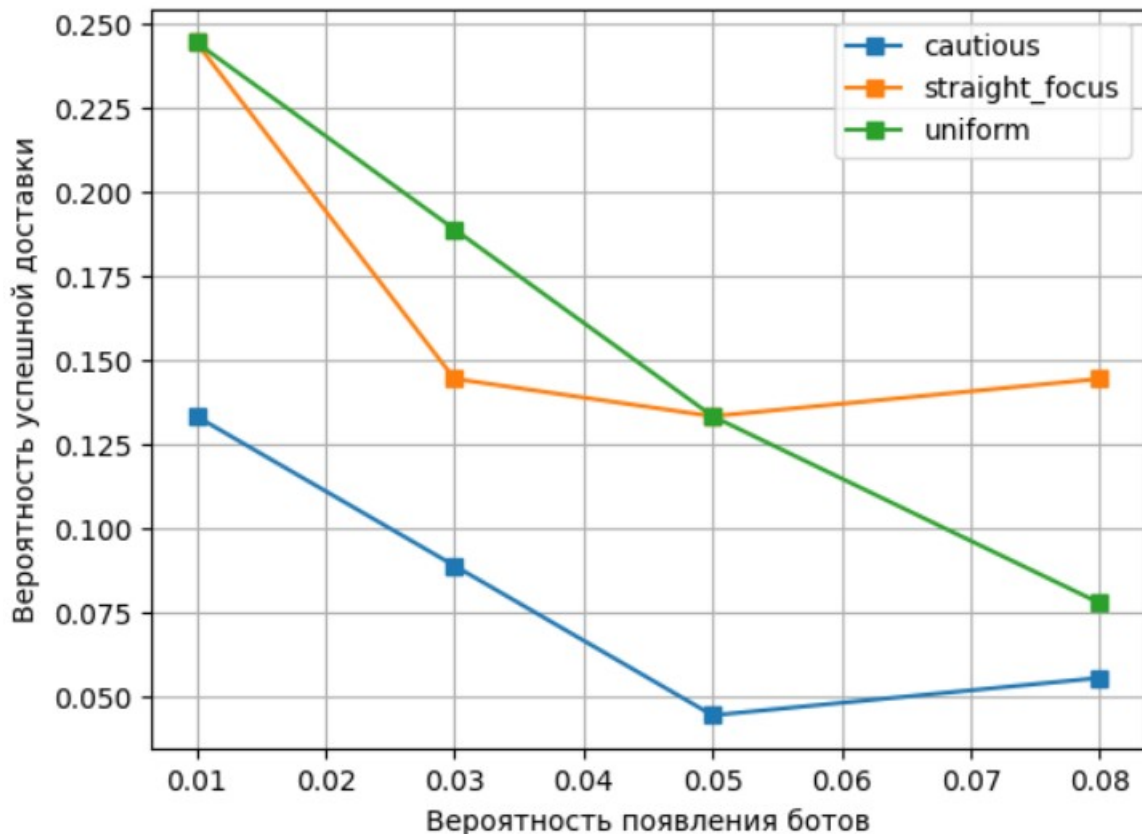


Рисунок 3 – График зависимости вероятности успешной доставки агентом от вероятности появления ботов

При вероятности 0.01 стратегии с преимущественно прямым направлением и равномерным показывают наилучший результат (~25% успешных доставок), в то время как аккуратная стратегия отстает с 13%. Это объясняется тем, что при низкой плотности трафика решительное движение вперед или случайное блуждание эффективнее излишне осторожного поведения с частыми остановками.

При вероятности 0.03 ситуация меняется: равномерное выходит в лидеры (18.5%), преимущественно прямое падает до 14%, а аккуратное падает до 8.5%. Равномерная стратегия начинает проявлять свою стабильность.

При вероятности 0.05 аккуратная достигает минимальных значений (менее 5%), в то время как остальные 2 стратегии показывают одинаковые результаты (~13%). Это подтверждает, что аккуратная стратегия становится практически непригодной при средней плотности трафика. Скорее всего, это вызвано увеличением трафика и боты начинают врезаться в агента, в связи с чем он не успевает дойти до точки финиша.

При вероятности 0.08 наблюдается интересный эффект: преимущественно прямая вновь становится лидером (14%), обгоняя равномерную (7.5%) и аккуратную (6%). Это может объясняться тем, что при экстремально высокой плотности ботов агрессивная стратегия успевает "проскочить" до появления критического количества препятствий, в то время как случайное блуждание и излишняя осторожность чаще приводят к столкновениям.

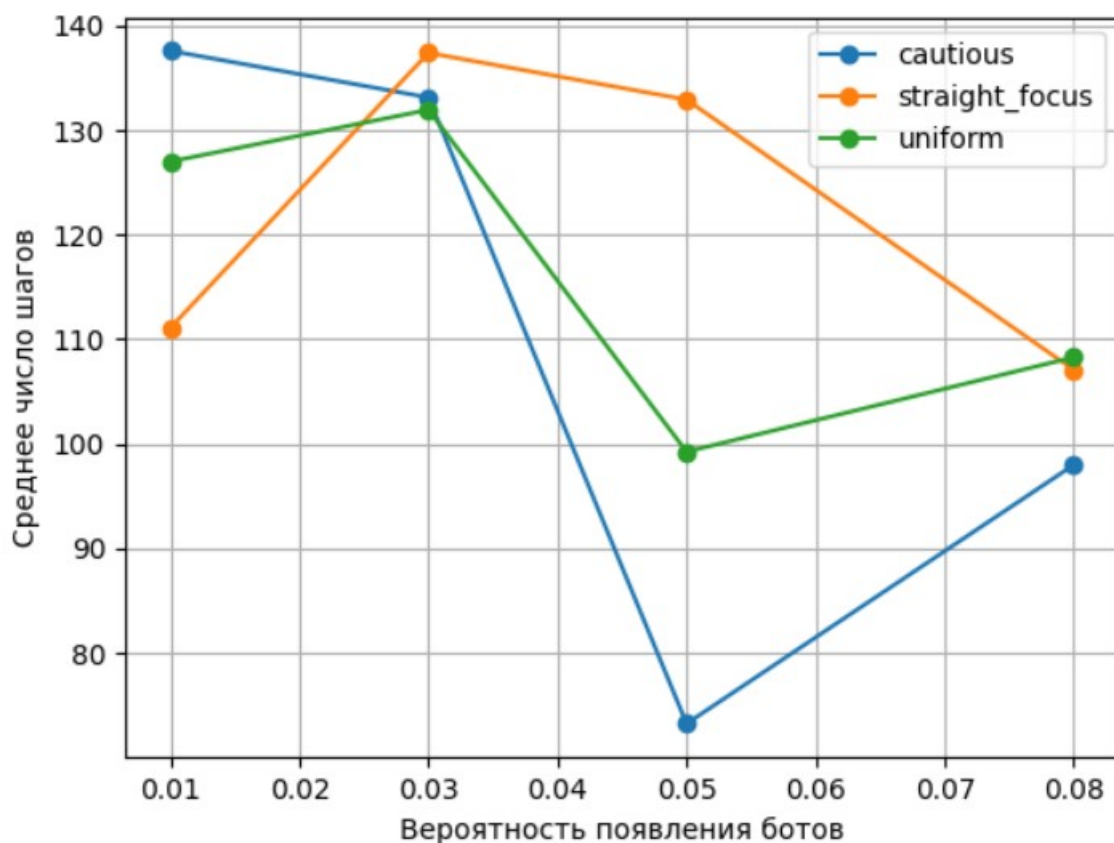


Рисунок 4 – График зависимости среднего количества шагов агента от вероятности появления ботов

График построен только для успешных эпизодов (агенты, которые доставили груз).

При вероятности 0.01 наибольшее среднее количество шагов у аккуратной стратегии (~138), наименьшее у стратегии с приоритетом прямо (~111). Это объясняется тем, что осторожная стратегия делает много остановок и разворотов, затягивая путь, даже когда ботов мало. Прямолинейная стратегия, наоборот, быстро движется к цели.

При вероятности 0.03 все три стратегии показывают близкие значения (~120-130 шагов). Это зона, где плотность трафика уже влияет на все стратегии примерно одинаково.

При вероятности 0.05 наблюдается резкое расхождение: стратегии с приоритетом прямо требует больше всего шагов (~130), равномерная стратегия около 100, а аккуратная падает ниже 80. Это связано с тем, что при высокой плотности ботов успешными остаются только короткие эпизоды. Осторожная стратегия, несмотря на остановки, либо быстро сталкивается с ботом (и не попадает в статистику успешных), либо очень быстро доходит до цели случайно. Прямолинейная стратегия, наоборот, упорно движется прямо, но из-за ботов её успешные эпизоды становятся длиннее. При этом вероятность успеха у прямолинейной и равномерной стратегий одинакова (~13%), а у аккуратной падает ниже 5%.

При вероятности 0.08 значения выравниваются: стратегия с приоритетом прямо и равномерная стратегия около 110, аккуратная около 100. Все стратегии в условиях плотного трафика показывают схожую эффективность по длине пути.

Общие выводы по результатам экспериментов:

- Стратегия cautious (аккуратная) показала наихудшие результаты на всем диапазоне как по вероятности успеха (от 5% до 13%), так и по количеству шагов (высокая вариативность). Частые остановки и развороты приводят к тому, что агент либо не успевает к цели, либо сталкивается с ботом. Приоритет остановкам вместо движения приводит к тому, что агент больше стоит на месте, чем движется к цели
- Стратегия straight_focus (приоритет прямо) демонстрирует высокую эффективность при низкой (0.01) и высокой (0.08) плотности трафика (25% и 14% успеха соответственно), но проигрывает uniform в среднем диапазоне (0.03-0.05). Однако на графике шагов видно, что при 0.05 успешные эпизоды у этой стратегии самые длинные (~130 шагов). Это может быть связано с тем, что агент упорно движется прямо, даже если это ведет в тупик, после чего вынужден разворачиваться и снова

двигаться прямо, создавая "маятниковое" движение. При высокой плотности ботов (0.08) такие длинные траектории становятся невозможными, и в статистику попадают только короткие успешные эпизоды, что выравнивает среднее количество шагов с другими стратегиями.

- Стратегия uniform (равномерная) наиболее стабильна в диапазоне 0.03-0.05 (18.5% и 13% успеха), что делает её предпочтительной при неопределенной дорожной обстановке. На графике шагов видно, что она занимает промежуточное положение между осторожной и прямолинейной стратегиями, что подтверждает её сбалансированный характер.
- Практическая рекомендация: для условий с низкой загруженностью дорог (0.01) оптимальна стратегия straight_focus, для непредсказуемого трафика (0.03-0.05) – uniform, а для экстремально высокой плотности ботов (0.08) вновь предпочтительна straight_focus. Стратегия cautious не рекомендуется к применению в данных условиях из-за низкой эффективности как по вероятности успеха, так и по длине маршрута.

Вывод

В ходе выполнения лабораторной работы была успешно реализована многоагентная среда с элементами неопределенности с применением многопроцессных вычислений на языке программирования Python.

Разработанная архитектура обеспечивает параллельное выполнение логики агента-доставщика и пула транспортных средств (ботов) в отдельных процессах с использованием примитивов синхронизации (Lock, Barrier, Manager), что гарантирует корректность обновления состояний и предотвращает гонки данных.

В результате проведенных экспериментов установлено, что выбор стратегии поведения агента оказывает существенное влияние на эффективность навигации:

- Стратегия `straight_focus` (приоритет прямого движения) демонстрирует наилучшие показатели при низкой (0.01) и высокой (0.08) вероятности появления ботов (25% и 14% успеха соответственно), однако при средней плотности трафика (0.05) успешные эпизоды становятся самыми длинными (~130 шагов), что связано с эффектом "маятникового" движения в тупиках.
- Стратегия `uniform` (равномерная) показывает наибольшую стабильность в среднем диапазоне вероятностей (0.03-0.05), достигая 18.5% и 13% успеха, что позволяет рекомендовать ее в качестве универсального решения для условий с неопределенной дорожной обстановкой.
- Стратегия `cautious` (аккуратная) оказалась наименее эффективной на всем диапазоне значений (успех от 5% до 13%), что объясняется приоритетом остановок перед движением: агент больше стоит на месте, чем движется к цели, что критично при ограниченном времени эпизода (250 шагов).

ИСХОДНЫЙ КОД ПРОГРАММЫ

[illegible]

```
import numpy as np
```

""Класс для навигации по карте""

ROAD VALUES = $\{0, 2, 3, 4\}$

```
def __init__(self, grid):
    self.grid = grid
```

```
def load_map(self, filename="citymap.txt"):
```

```

"""Загружает матрицу карты из текстового файла"""
with open(filename, "r") as f:
    rows = [list(map(int, line.split())) for line in f if line.strip()]
self.grid = np.array(rows)
return self.grid

```

```
def find_start_and_goal(self):
```

""""Находит на карте стартовую точку (2) и цель (4)""""

```

start_points = []
goal_pos = None

for i in range(len(self.grid)):
    for j in range(len(self.grid[0])):
        if self.grid[i][j] == 2:
            start_points.append((i, j))
        elif self.grid[i][j] == 4:
            goal_pos = (i, j)

return start_points, goal_pos

def get_valid_neighbors(self, pos):
    """Возвращает список координат соседних клеток, доступных для перемещения"""
    x, y = pos
    h = len(self.grid)
    w = len(self.grid[0])
    candidates = [(x - 1, y), (x + 1, y), (x, y - 1), (x, y + 1)]
    return [(nx, ny) for nx, ny in candidates
            if 0 <= nx < h and 0 <= ny < w and self.grid[nx][ny] in self.ROAD_VALUES]

def classify_cell(self, pos, prev_pos):
    """Определяет тип текущей клетки: 'start', 'dead_end', 'intersection', 'straight'"""
    if prev_pos is None:
        return 'start'

    valid = self.get_valid_neighbors(pos)
    forward_options = [n for n in valid if n != prev_pos]

    if len(forward_options) == 0:
        return 'dead_end'
    elif len(forward_options) >= 2:
        return 'intersection'
    else:
        return 'straight'

def resolve_direction(self, pos, prev_pos, choice):
    """Преобразует направление в координаты следующей клетки"""
    if choice == 'stop':
        return pos

    h = len(self.grid)
    w = len(self.grid[0])

    dx, dy = pos[0] - prev_pos[0], pos[1] - prev_pos[1]

    directions = {
        'straight': (dx, dy),
        'back': (-dx, -dy),
        'left': (-dy, dx),
        'right': (dy, -dx)
    }

    target_vec = directions.get(choice)
    if target_vec is None:
        raise ValueError("Недопустимое значение направления: " + choice)

    nx, ny = pos[0] + target_vec[0], pos[1] + target_vec[1]

```



```

    if 0 <= nx < h and 0 <= ny < w and self.grid[nx][ny] in self.ROAD_VALUES:
        return (nx, ny)
    return pos

```

simulator.py

```
import multiprocessing as mp
```

```
import random
```

```
from navigation import Navigation
```

```
class Simulator:
```

```
    """Многопроцессный симулятор движения агента и ботов"""
```

```
    def __init__(self, grid, goal_pos, spawn_prob, strategy_weights):
```

```
        self.grid = grid
```

```
        self.nav = Navigation(grid)
```

```
        self.goal_pos = goal_pos
```

```
        self.spawn_prob = spawn_prob
```

```
        self.strategy_weights = strategy_weights
```

```
    def init_shared_state(self, start_pos):
```

```
        """Создает разделяемое состояние для синхронизации процессов"""
```

```
        manager = mp.Manager()
```

```
        shared = manager.dict()
```

```
        shared['grid'] = [list(row) for row in self.grid]
```

```
        shared['agent'] = manager.dict({
```

```
            'pos': start_pos,
```

```
            'prev': start_pos,
```

```
            'steps': 0,
```

```
            'stop_next_reverse': False,
```

```
            'alive': True
```

```
        })
```

```
        shared['bots'] = []
```

```
        shared['goal_pos'] = self.goal_pos
```

```
        shared['episode_done'] = False
```

```
        shared['success'] = False
```

```
        shared['collided'] = False
```

```
        return shared
```

```
    def check_collisions(self, shared):
```

```
        """Проверяет столкновения между агентом и ботами, а также ботов между собой"""
```

```
        agent_pos = shared['agent']['pos']
```

```
        agent_prev = shared['agent']['prev']
```

```
        current_bots = shared['bots']
```

```
        # Столкновение агента с ботом (на одной клетке)
```

```
        for bot in current_bots:
```

```
            if bot['pos'] == agent_pos:
```

```
                shared['collided'] = True
```

```
                shared['episode_done'] = True
```

```
                shared['success'] = False
```

```
                return
```

```
        # агент и бот разминутись (поменялись местами)
```

```
        for bot in current_bots:
```

```
            bot_prev = bot['prev']
```

```
            bot_current = bot['pos']
```

```
        # Если агент был на месте, где сейчас бот, И бот был на месте, где сейчас агент
```

```
        if bot_prev == agent_pos and bot_current == agent_prev:
```

```
            shared['collided'] = True
```

```

    shared['episode_done'] = True
    shared['success'] = False
    return

# Столкновения ботов между собой (на одной клетке)
pos_counts = {}
for bot in current_bots:
    p = bot['pos']
    pos_counts[p] = pos_counts.get(p, 0) + 1

# боты разминулись (поменялись местами)
for i, bot1 in enumerate(current_bots):
    for bot2 in current_bots[i+1:]:
        if bot1['prev'] == bot2['pos'] and bot2['prev'] == bot1['pos']:
            # Удаляем обоих ботов
            pos_counts[bot1['pos']] = 0
            pos_counts[bot2['pos']] = 0

surviving_bots = [bot for bot in current_bots if pos_counts[bot['pos']] == 1]
shared['bots'] = surviving_bots

# Проверка достижения цели
if agent_pos == shared['goal_pos']:
    shared['episode_done'] = True
    shared['success'] = True

def agent_worker(self, shared, lock, barrier):
    """Процесс управления агентом-доставщиком"""
    directions = ['straight', 'right', 'left', 'stop']

    while True:
        lock.acquire()
        if shared['episode_done']:
            lock.release()
            break

        agent = shared['agent']
        pos = agent['pos']
        prev = agent['prev']
        grid = shared['grid']

        # Разворот после остановки
        if agent['stop_next_reverse']:
            direction = 'back'
            agent['stop_next_reverse'] = False
            next_pos = self.nav.resolve_direction(pos, prev, direction)

        # Первый шаг (позиция совпадает с предыдущей)
        elif prev == pos:
            valid = self.nav.get_valid_neighbors(pos)
            next_pos = random.choice(valid) if valid else pos

        else:
            cell_type = self.nav.classify_cell(pos, prev)

            if cell_type == 'dead_end':
                direction = 'back'
                next_pos = self.nav.resolve_direction(pos, prev, direction)

```

```

elif cell_type == 'intersection':
    direction = random.choices(directions, weights=self.strategy_weights, k=1)[0]
    if direction == 'stop':
        agent['stop_next_reverse'] = True
        next_pos = pos
    else:
        next_pos = self.nav.resolve_direction(pos, prev, direction)

else: # straight
    direction = 'straight'
    next_pos = self.nav.resolve_direction(pos, prev, direction)

agent['prev'] = pos
agent['pos'] = next_pos
agent['steps'] = agent['steps'] + 1

lock.release()

try:
    barrier.wait(timeout=10.0)
except Exception:
    break

def bot_worker(self, shared, lock, barrier):
    """Процесс управления пулом транспортных средств"""
    while True:
        lock.acquire()
        if shared['episode_done']:
            lock.release()
            break

        grid = shared['grid']
        h = len(grid)
        w = len(grid[0])

        # Поиск всех точек спавна (значение 3 на карте)
        spawn_points = [(r, c) for r in range(h) for c in range(w) if grid[r][c] == 3]

        current_bots = shared['bots']

        # Спавн нового бота
        if random.random() < self.spawn_prob and spawn_points:
            spawn_pos = random.choice(spawn_points)
            current_bots.append({
                'pos': spawn_pos,
                'prev': spawn_pos,
                'lifetime': random.randint(15, 150),
                'stop_next_reverse': False,
                'alive': True
            })

        active_bots = []
        for bot in current_bots:
            if not bot.get('alive', True):
                continue

            bot['lifetime'] = bot['lifetime'] - 1
            if bot['lifetime'] <= 0:
                continue

```

```

pos = bot['pos']
prev = bot['prev']

# Разворот после остановки
if bot['stop_next_reverse']:
    direction = 'back'
    bot['stop_next_reverse'] = False
    next_pos = self.nav.resolve_direction(pos, prev, direction)

elif prev == pos:
    valid = self.nav.get_valid_neighbors(pos)
    next_pos = random.choice(valid) if valid else pos

else:
    cell_type = self.nav.classify_cell(pos, prev)

    if cell_type == 'dead_end':
        direction = 'back'
        next_pos = self.nav.resolve_direction(pos, prev, direction)

    elif cell_type == 'intersection':
        valid = self.nav.get_valid_neighbors(pos)
        forward = [n for n in valid if n != prev]
        if len(forward) > 0 and random.random() < 0.1:
            bot['stop_next_reverse'] = True
            next_pos = pos
        else:
            direction = random.choice(['straight', 'left', 'right'])
            next_pos = self.nav.resolve_direction(pos, prev, direction)

    else: # straight
        direction = 'straight'
        next_pos = self.nav.resolve_direction(pos, prev, direction)

bot['prev'] = pos
bot['pos'] = next_pos
active_bots.append(bot)

shared['bots'] = active_bots
lock.release()

try:
    barrier.wait(timeout=10.0)
except Exception:
    break

def run_episode(self, start_pos):
    """Запускает один эпизод симуляции"""
    shared = self.init_shared_state(start_pos)
    lock = mp.Lock()
    barrier = mp.Barrier(3)

    agent_proc = mp.Process(target=self.agent_worker, args=(shared, lock, barrier))
    bot_proc = mp.Process(target=self.bot_worker, args=(shared, lock, barrier))

    agent_proc.start()
    bot_proc.start()

```

```

max_steps = 250

for _ in range(max_steps):
    try:
        barrier.wait(timeout=10.0)
    except Exception:
        shared['episode_done'] = True
        break

    lock.acquire()
    self.check_collisions(shared)
    lock.release()

    if shared['episode_done']:
        break

agent_proc.join(timeout=5)
bot_proc.join(timeout=5)

if agent_proc.is_alive():
    agent_proc.terminate()
if bot_proc.is_alive():
    bot_proc.terminate()

return {
    'steps': shared['agent']['steps'],
    'success': shared['success'],
    'collided': shared['collided']
}

```

experiment.py

import csv

from simulator import Simulator

class Experiment:

"""Проведение экспериментов с разными параметрами"""

def __init__(self, grid, goal_pos, start_points, spawn_rates, strategies, episodes_per_config=30):

self.grid = grid

self.goal_pos = goal_pos

self.start_points = start_points

self.spawn_rates = spawn_rates

self.strategies = strategies

self.episodes_per_config = episodes_per_config

def run_batch(self, spawn_rate, strategy_name, strategy_weights):

"""Выполняет серию эпизодов для заданной конфигурации"""

results = []

sim = Simulator(self.grid, self.goal_pos, spawn_rate, strategy_weights)

for _ in range(self.episodes_per_config):

for start_pos in self.start_points:

outcome = sim.run_episode(start_pos)

outcome['spawn_rate'] = spawn_rate

outcome['strategy'] = strategy_name

outcome['start_pos'] = start_pos

results.append(outcome)

return results

```

def save_to_csv(self, data, filename="experiment_results.csv"):
    """Сохраняет результаты в CSV файл"""
    if not data:
        return

    fields = data[0].keys()
    with open(filename, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        writer.writerows(data)
    print("Результаты сохранены в " + filename)

def run_all(self, output_file="experiment_results.csv"):
    """Запускает все эксперименты"""
    all_results = []

    for rate in self.spawn_rates:
        for name, weights in self.strategies.items():
            print(f"Запуск конфигурации: скорость={rate} стратегия={name}")
            batch = self.run_batch(rate, name, weights)
            all_results.extend(batch)

    self.save_to_csv(all_results, output_file)

```

visualize.py

```

import csv
import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as anim
import multiprocessing as mp
from matplotlib.colors import ListedColormap, BoundaryNorm
from simulator import Simulator

class Visualizer:
    """Визуализация результатов и анимации"""

    def __init__(self, grid):
        self.grid = grid

    def load_results(self, filename="experiment_results.csv"):
        """Загружает данные эксперимента из CSV файла"""
        data = []
        with open(filename, "r") as f:
            reader = csv.DictReader(f)
            for row in reader:
                row['steps'] = int(row['steps'])
                row['success'] = row['success'] == 'True'
                row['spawn_rate'] = float(row['spawn_rate'])
                data.append(row)
        return data

    def plot_metrics(self, data, output_dir="results"):
        """Строит графики зависимости шагов и вероятности успеха от скорости спавна"""
        os.makedirs(output_dir, exist_ok=True)

        strategies = sorted(list(set(r['strategy'] for r in data)))
        rates = sorted(list(set(r['spawn_rate'] for r in data)))

        avg_steps_data = {}

```

```

success_probs_data = {}

for strat in strategies:
    avg_steps_data[strat] = []
    success_probs_data[strat] = []

    for r in rates:
        subset = [d for d in data if d['strategy'] == strat and d['spawn_rate'] == r]
        if subset:
            successful = [d for d in subset if d['success']]
            avg = np.mean([d['steps'] for d in successful]) if successful else None
            prob = len(successful) / len(subset)
            avg_steps_data[strat].append(avg)
            success_probs_data[strat].append(prob)
        else:
            avg_steps_data[strat].append(None)
            success_probs_data[strat].append(np.nan)

# График среднего числа шагов
plt.figure()
for strat in strategies:
    # Фильтруем None значения
    clean_rates = []
    clean_steps = []
    for r, s in zip(rates, avg_steps_data[strat]):
        if s is not None and not np.isnan(s):
            clean_rates.append(r)
            clean_steps.append(s)

    if clean_rates: # рисуем только если есть данные
        plt.plot(clean_rates, clean_steps, marker='o', label=strat)
plt.xlabel("Вероятность появления ботов")
plt.ylabel("Среднее число шагов")
plt.legend()
plt.grid(True)
plt.savefig(os.path.join(output_dir, "steps_vs_spawn.png"))
plt.close()

# График вероятности успеха
plt.figure()
for strat in strategies:
    plt.plot(rates, success_probs_data[strat], marker='s', label=strat)
plt.xlabel("Вероятность появления ботов")
plt.ylabel("Вероятность успешной доставки")
plt.legend()
plt.grid(True)
plt.savefig(os.path.join(output_dir, "success_vs_spawn.png"))
plt.close()

print("Графики сохранены в директории " + output_dir)

def generate_animation(self, start_pos, goal_pos, spawn_prob, strategy_weights, filename="episode_animation.gif"):
    """Создает анимацию движения агента и ботов"""
    frames = []
    sim = Simulator(self.grid, goal_pos, spawn_prob, strategy_weights)
    shared = sim.init_shared_state(start_pos)

    lock = mp.Lock()
    barrier = mp.Barrier(3)

```

```

agent_proc = mp.Process(target=sim.agent_worker, args=(shared, lock, barrier))
bot_proc = mp.Process(target=sim.bot_worker, args=(shared, lock, barrier))

agent_proc.start()
bot_proc.start()

max_steps = 200

for _ in range(max_steps):
    try:
        barrier.wait(timeout=10.0)
    except Exception:
        break

    lock.acquire()
    sim.check_collisions(shared)

    base_grid = np.array(shared['grid']).copy()
    ax, ay = shared['agent']['pos']
    if 0 <= ax < len(self.grid) and 0 <= ay < len(self.grid[0]):
        base_grid[ax, ay] = 5

    for bot in shared['bots']:
        if bot.get('alive', True):
            bx, by = bot['pos']
            if 0 <= bx < len(self.grid) and 0 <= by < len(self.grid[0]):
                base_grid[bx, by] = 6

    frames.append(base_grid)

    if shared['episode_done']:
        lock.release()
        break
    lock.release()

agent_proc.join(timeout=5)
bot_proc.join(timeout=5)

if not frames:
    print("Кадры не записаны")
    return

fig, ax = plt.subplots(figsize=(6, 6))
cmap = ListedColormap(["#777580", "#1a1a2e", "#00ff26", "#bac43a", "#ff8235", "#ff0000", "#32cfff"])
bounds = range(8)
norm = BoundaryNorm(bounds, len(cmap.colors))

im = ax.imshow(frames[0], cmap=cmap, norm=norm, origin="upper")
ax.set_xticks([])
ax.set_yticks([])

def update(frame):
    im.set_array(frame)
    return [im]

ani = anim.FuncAnimation(fig, update, frames=frames, interval=100, repeat=False)
ani.save(filename, writer="pillow", fps=10)
print("Анимация сохранена: " + filename)

```


main.py

```
from navigation import Navigation
from experiment import Experiment
from visualize import Visualizer

def main():
    # Общая конфигурация
    MAP_FILE = "citymap.txt"
    START_POS = None          # None = найдет автоматически первую точку с цифрой 2
    GOAL_POS = None           # None = найдет автоматически точку с цифрой 4
    SPAWN_RATES = [0.01, 0.03, 0.05, 0.08]
    STRATEGIES = {
        'uniform': [0.25, 0.25, 0.25, 0.25],
        'straight_focus': [0.50, 0.20, 0.20, 0.10],
        'cautious': [0.10, 0.10, 0.10, 0.70]
    }
    EPISODES_PER_CONFIG = 30
    EXPERIMENT_RESULT_FILE = "experiment_results.csv"

    # Загружаем карту
    nav = Navigation(None)
    grid = nav.load_map(MAP_FILE)

    print(f"Карта загружена. Размер: {grid.shape}")

    # Автоматически находим стартовые точки и цель на карте
    START_POS, GOAL_POS = nav.find_start_and_goal()

    if not START_POS:
        print("Ошибка: на карте нет стартовой точки (цифра 2)")
        return
    if not GOAL_POS:
        print("Ошибка: на карте нет цели (цифра 4)")
        return

    print(f"Найдены стартовые точки: {START_POS}")
    print(f"Найдена цель: {GOAL_POS}")

    # Запуск экспериментов
    experiment = Experiment(grid, GOAL_POS, START_POS, SPAWN_RATES, STRATEGIES,
        episodes_per_config=EPISODES_PER_CONFIG)
    experiment.run_all(EXPERIMENT_RESULT_FILE)

    # Визуализация результатов
    visualizer = Visualizer(grid)
    data = visualizer.load_results(EXPERIMENT_RESULT_FILE)
    if data:
        visualizer.plot_metrics(data)
    else:
        print("Нет данных для визуализации")

if __name__ == "__main__":
    main()

run_animation.py
from navigation import Navigation
from visualize import Visualizer
```

```

def main():
    """
    Простой запуск анимации с выводом результата в консоль.
    """

    MAP_FILE = "citymap.txt"
    START_POS = None          # None = найдет автоматически первую точку с цифрой 2
    GOAL_POS = None          # None = найдет автоматически точку с цифрой 4
    SPAWN_PROB = 0.1         # вероятность появления ботов (0.01, 0.03, 0.05, 0.08)
    STRATEGY_WEIGHTS = [0.25, 0.25, 0.25, 0.25] # uniform
    OUTPUT_FILE = "episode_animation.gif"

    # Загружаем карту
    nav = Navigation(None)
    grid = nav.load_map(MAP_FILE)
    print(f"Карта загружена. Размер: {grid.shape}")

    # Автоматически находим старт и цель
    START_POS, GOAL_POS = nav.find_start_and_goal()

    # Проверки
    if START_POS is None:
        print("Ошибка: на карте нет стартовой точки (цифра 2)")
        return
    if GOAL_POS is None:
        print("Ошибка: на карте нет цели (цифра 4)")
        return
    start = START_POS[0]
    # Вывод настроек
    print(f"Стартовая позиция: {start}")
    print(f"Цель: {GOAL_POS}")
    print(f"Вероятность спавна ботов: {SPAWN_PROB}")
    print(f"Стратегия: {STRATEGY_WEIGHTS}")
    print(f"Выходной файл: {OUTPUT_FILE}")

    # Запускаем анимацию
    visualizer = Visualizer(grid)

    print("Запуск симуляции...")
    visualizer.generate_animation(
        start_pos=start,
        goal_pos=GOAL_POS,
        spawn_prob=SPAWN_PROB,
        strategy_weights=STRATEGY_WEIGHTS,
        filename=OUTPUT_FILE
    )

if __name__ == "__main__":
    main()

```